PROGRESS II
KONTROL MOTOR NIRKABEL DENGAN BLYNK

2.1 Tujuan Modul

Tahap Progress 2 bertujuan untuk mengimplementasikan sistem kendali motor secara nirkabel (*wireless*) menggunakan jaringan Wi-Fi dan *platform* IoT (Blynk). Pengujian ini difokuskan pada kemampuan mikrokontroler ESP32 dalam menerima instruksi jarak jauh melalui antarmuka *smartphone* secara *real-time*, memvalidasi kestabilan protokol komunikasi TCP/IP, serta memastikan integrasi logika PWM dengan *Virtual Pin* (Datastream) beroperasi tanpa adanya kendala atau penundaan (*delay*).

2.2 Skema Rangkaian

Skema pengkabelan pada tahap ini tidak mengalami perubahan fisik dari tahap Progress 1. ESP32 tetap dihubungkan ke modul *driver* motor L298N dengan pemetaan:

- Pin 27 (motor1Pin1): Dihubungkan ke pin IN1 pada *driver* motor (mengatur arah putaran).
- Pin 26 (motor1Pin2): Dihubungkan ke pin IN2 pada *driver* motor (mengatur arah putaran).
- Pin 12 (enable1Pin): Dihubungkan ke pin ENA pada *driver* motor untuk menerima sinyal kontrol kecepatan PWM.

2.3 Konsep & Logika Sistem

Pada tahap ini, instruksi berbasis antarmuka Serial sepenuhnya digantikan oleh komunikasi *Cloud* ke *server* Blynk. Sistem menggunakan konsep Virtual Pin, yakni jalur data virtual yang menjembatani komponen antarmuka pengguna (*User Interface*) di layar *smartphone* dengan variabel di dalam memori ESP32. Mikrokontroler diprogram menggunakan paradigma *Event-Driven* melalui fungsi BLYNK_WRITE(). Saat pengguna berinteraksi dengan aplikasi (misalnya menggeser *slider* kecepatan), server akan mengirimkan interupsi (*interrupt*) ke ESP32 yang langsung dikonversi menjadi sinyal PWM fisik (0-255) menuju *driver* motor.

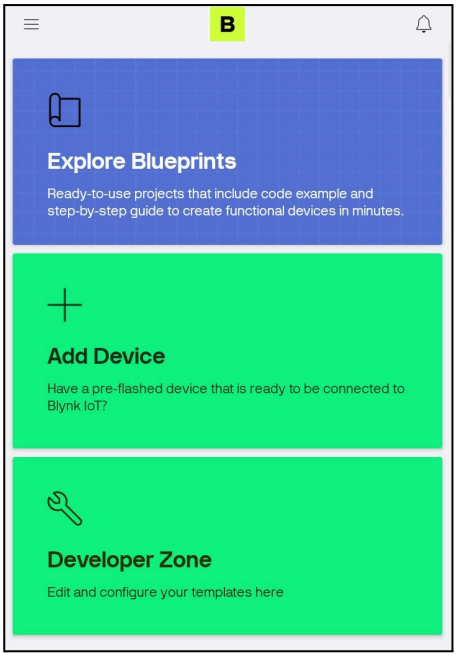
2.4 Pembuatan Device dan Otentikasi

Karena *Template* dan jalur *Datastream* (V0 hingga V8) telah dikonfigurasi pada tahap persiapan dasar (SETUP), langkah selanjutnya adalah menginstansiasi

Template tersebut menjadi sebuah *Device* (perangkat keras virtual) dan mengatur tata letak antarmuka pengguna (*User Interface*) di aplikasi *smartphone*.

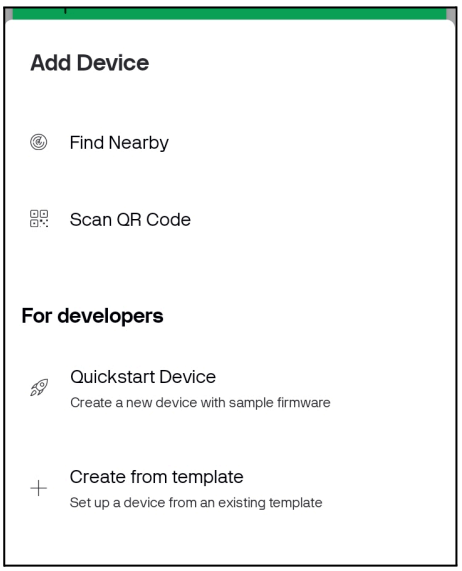
a. Instansiasi Device Baru

1. Buka aplikasi Blynk IoT di *smartphone* Anda.
2. Pada halaman utama (Beranda), tekan menu Add Device.



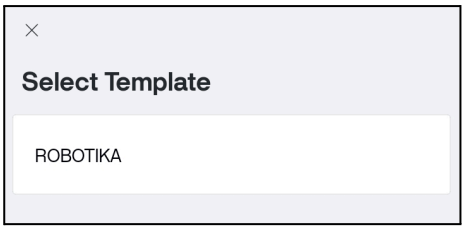
Gambar 1. Menu utama (Beranda) aplikasi Blynk untuk menambahkan perangkat.

3. Pilih opsi From Template.



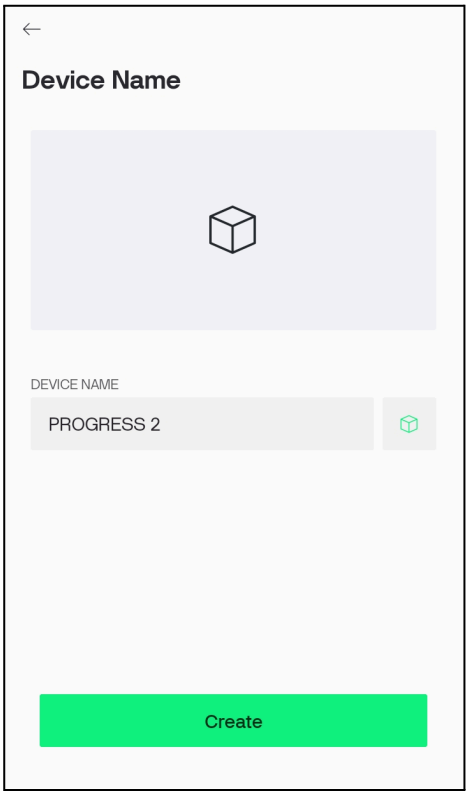
Gambar 2. Opsi penambahan perangkat menggunakan metode *Template*.

4. Pilih *Template* yang telah dibuat sebelumnya (misalnya: Template "ROBOTIKA").



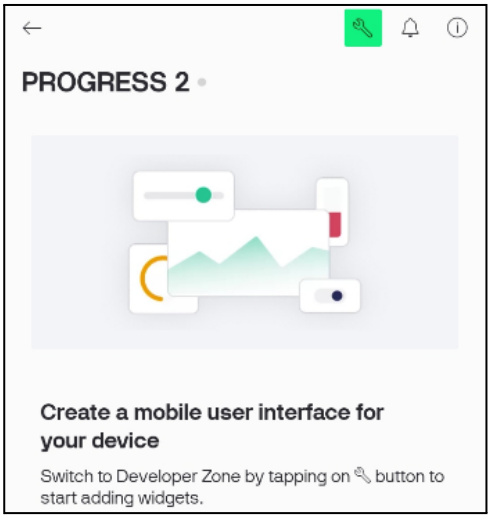
Gambar 3. Pemilihan *Template* yang telah dikonfigurasi pada tahap Setup.

5. Beri nama perangkat (contoh: "PROGRESS 2"), lalu klik Create.



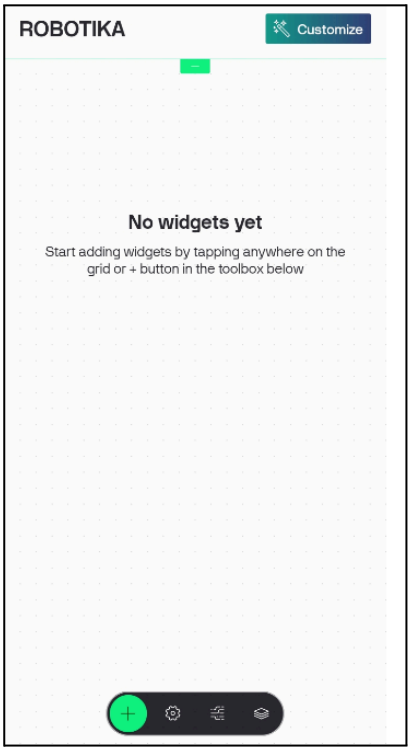
Gambar 4. Pemberian nama identitas pada *Device* baru.

6. Selanjutnya, perangkat (*device*) baru yang masih kosong akan berhasil dibuat.



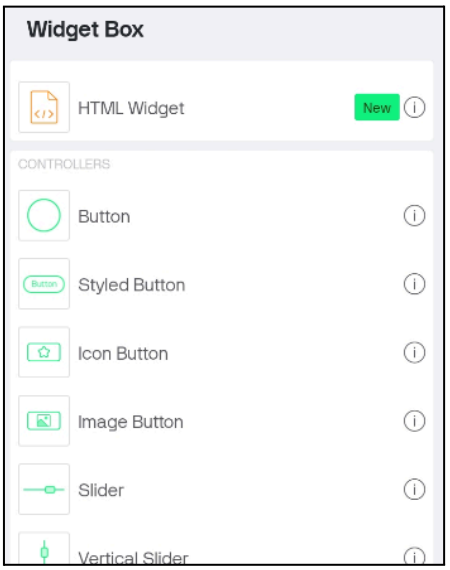
Gambar 5. Tampilan awal antarmuka *Device* "PROGRESS 2" yang masih kosong.

7. Langkah selanjutnya, klik logo + (Plus) berwarna hijau untuk menambahkan komponen antarmuka.



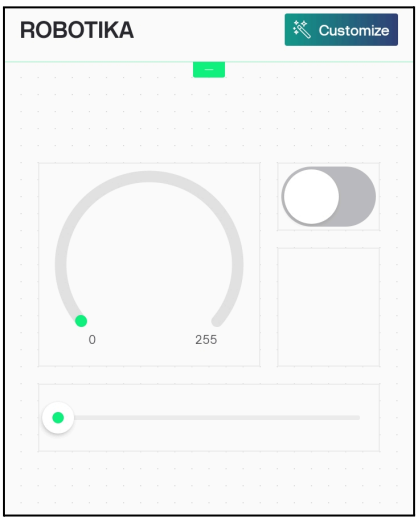
Gambar 6. Kanvas *Developer Zone* untuk merancang antarmuka pengguna.

8. Selanjutnya pilih 4 *widget*, yakni Switch, Slider, Gauge, dan juga Value Display.



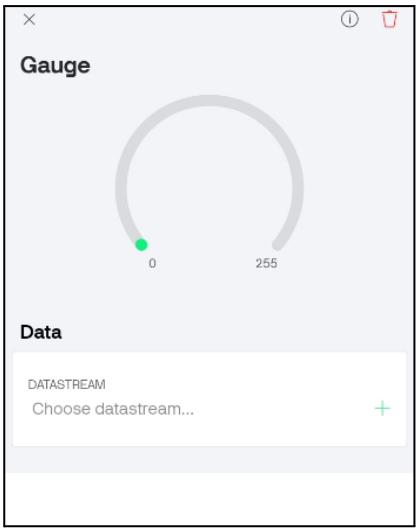
Gambar 7. Menu *Widget Box* yang berisi daftar komponen visual.

9. Selanjutnya atur dengan menekan lalu menahan (*hold and drag*) sehingga komponen tersebut berbentuk seperti gambar berikut atau tersusun sesuai dengan keinginan Anda.



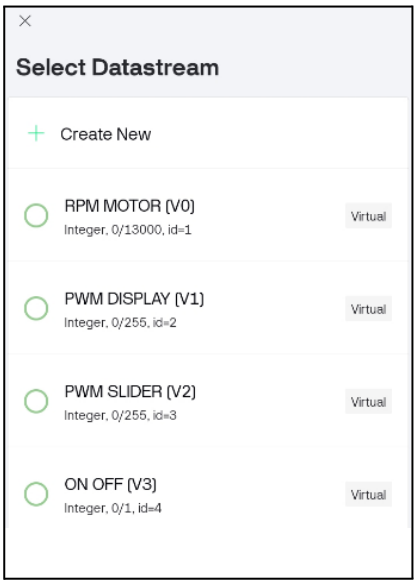
Gambar 8. Proses penyesuaian tata letak (*layouting*) *widget* pada kanvas.

10. Selanjutnya, tekan salah satu *widget* (contohnya *Gauge*) untuk diisi dengan *Datastream* dengan menekan tanda + (Plus) pada bagian *Choose Datastream*.



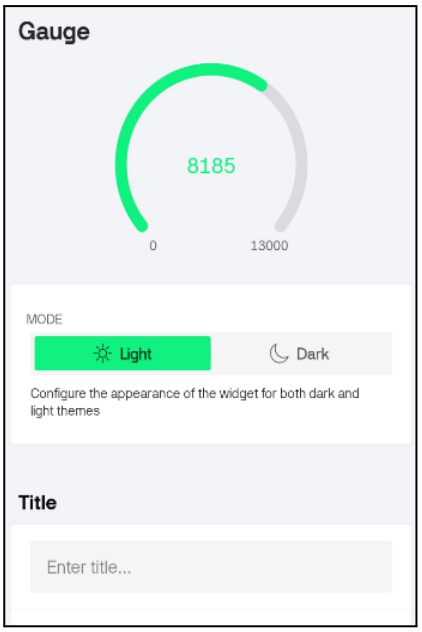
Gambar 9. Menu pengaturan parameter pada komponen *widget*.

11. Selanjutnya akan ditampilkan *Datastream* yang telah dibuat pada tahap SETUP. Pilih sesuai dengan yang diinstruksikan: Untuk *Gauge* pertama isi dengan *Datastream* V0 (RPM MOTOR), untuk *Slider* isi dengan V2 (PWM SLIDER), untuk *Value Display* isi dengan V1 (PWM DISPLAY), dan untuk *Switch* isi dengan V3 (ON OFF).



Gambar 10. Pemetaan setiap komponen *widget* menuju jalur *Datastream* yang relevan.

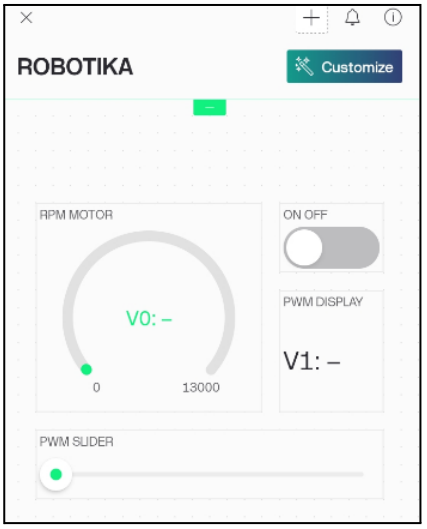
12. Setelah *widget* diisi dengan *Datastream*, selanjutnya edit namanya dengan menekan tab Design. Sesuaikan namanya dengan nama pada *Datastream* agar informatif.



Gambar 11. Penyesuaian nama *widget* pada tab pengaturan *Design*.

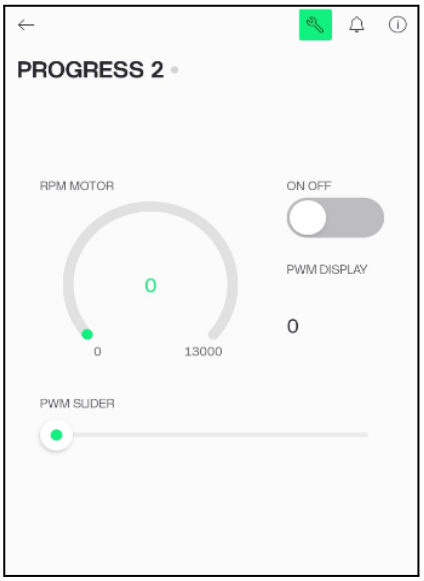
Lakukan langkah penyesuaian nama dan Datastream pada semua widget sampai dengan selesai.

13. Setelah itu, tampilan tata letak akhirnya akan menjadi sebagai berikut



Gambar 12. Hasil akhir rancangan antarmuka pengguna di mode *Developer Zone*.

14. Selanjutnya, tekan tombol panah mundur/silang (kembali) untuk melihat hasil akhir dari *device* di mode operasional pengguna seperti gambar berikut:

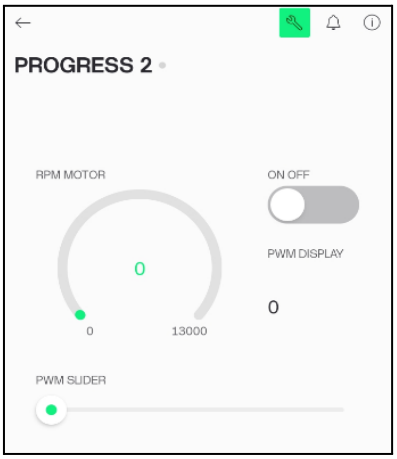


Gambar 13. Tampilan operasional aplikasi Blynk siap pakai.

b. Template Id, Auth Token, dan Template name

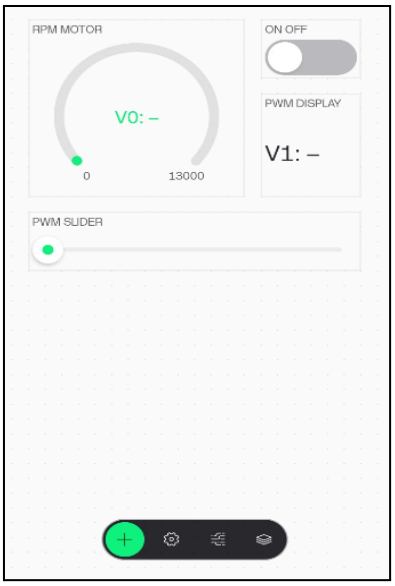
Setelah *Device* terbentuk, langkah krusial selanjutnya adalah mendapatkan token otentikasi agar alat (ESP32) dapat dikenali oleh server.

1. Tekan tombol berbentuk seperti kunci pas untuk masuk ke pengaturan perangkat.



Gambar 14. Indikator *widget* yang masih bernilai 0 sebelum dihubungkan ke alat fisik.

2. Setelah masuk ke mode pengeditan (Developer), klik tab ikon gerigi (Settings) pada bagian bawah layar.



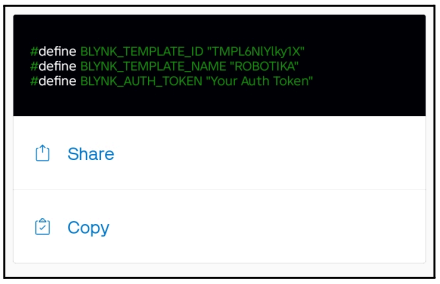
Gambar 15. Menuju mode pengaturan (*Settings*) perangkat.

3. Selanjutnya, pilih opsi Code untuk melihat kode otentikasi unik untuk perangkat Blynk tersebut.



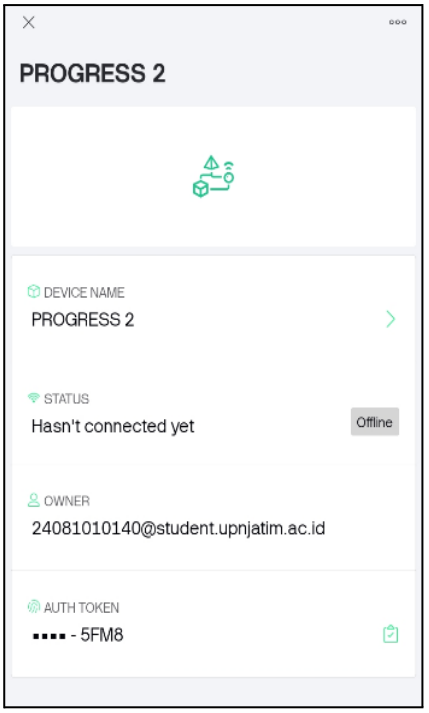
Gambar 16. Menu *Settings* untuk melihat informasi perangkat.

4. Selanjutnya, pilih opsi Code untuk melihat kode otentikasi unik untuk perangkat Blynk tersebut.



Gambar 17. Menu yang menampilkan kode rahasia

5. Anda juga dapat kembali ke tampilan operasional aplikasi, lalu mengeklik ikon informasi (i), maka Auth Token akan ditampilkan.



Gambar 18. Tampilan profil *Device* dan salinan kredensial (Auth Token) yang siap disisipkan.

Salin baris kode kredensial tersebut, lalu gabungkan dan *paste* (tempel) di baris paling atas pada aplikasi Arduino IDE Anda (pada bagian Kredensial Blynk & WiFi).

2.5 Source Code dan Penjelasan Logika Program

```
// PROGRESS 2 : KONTROL MOTOR MURNI BLYNK
// Fitur: ON/OFF (V3) dan PWM Slider (V2), Display PWM (V1) & RPM (V0)

// --- KREDENSIAL BLYNK & WIFI ---
#define BLYNK_TEMPLATE_ID "ISI_DENGAN_TEMPLATE_ID_ANDA"
#define BLYNK_TEMPLATE_NAME "ISI_DENGAN_NAMA_TEMPLATE_ANDA"
#define BLYNK_AUTH_TOKEN "ISI_DENGAN_AUTH_TOKEN_DEVICE_BARU"
#define BLYNK_PRINT Serial

#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>

// Masukkan SSID dan Password Wi-Fi/Tethering Anda
char ssid[] = "NAMA_WIFI_ANDA";
char pass[] = "PASSWORD_WIFI_ANDA";

// --- KONFIGURASI PIN ---
int ledPin = 2;          // Indikator LED internal ESP32
int motor1Pin1 = 27;     // Pin IN1 Driver Motor
int motor1Pin2 = 26;     // Pin IN2 Driver Motor
int enable1Pin = 12;     // Pin ENA (PWM) Driver Motor

// --- PENGATURAN PWM ---
const int freq = 30000;      // Frekuensi 30 kHz
const int resolution = 8;    // Resolusi 8-bit (0-255)
int dutyCycle = 0;           // Kecepatan awal (mati)
bool isMotorRunning = false; // Status motor

// --- FUNGSI UPDATE TAMPILAN BLYNK ---
// Memperbarui nilai Gauge dan Display di HP
void updateBlynkDisplay() {
  Blynk.virtualWrite(V1, dutyCycle); // Update Gauge PWM (0-255)
  Blynk.virtualWrite(V2, dutyCycle); // Update posisi Slider agar sinkron

  // Simulasi perhitungan RPM kasar berdasarkan rasio PWM (Maks 13000)
  int simulasiRPM = map(dutyCycle, 0, 255, 0, 13000);
  if(!isMotorRunning) simulasiRPM = 0;
  Blynk.virtualWrite(V0, simulasiRPM); // Update Gauge RPM
}

// --- FUNGSI HARDWARE MOTOR ---
void applyMotor() {
```

```
  if (isMotorRunning) {
    digitalWrite(motor1Pin1, HIGH);
    digitalWrite(motor1Pin2, LOW);
    ledcWrite(enable1Pin, dutyCycle);
  } else {
    digitalWrite(motor1Pin1, LOW);
    digitalWrite(motor1Pin2, LOW);
    ledcWrite(enable1Pin, 0);
  }
  updateBlynkDisplay();
}

// --- LOGIKA BLYNK ---

// Menerima perintah dari Tombol ON/OFF di HP (Datastream V3)
BLYNK_WRITE(V3) {
  int pinValue = param.asInt(); // Baca status tombol (1 = ON, 0 = OFF)

  if (pinValue == 1) {
    isMotorRunning = true;
    if (dutyCycle == 0) dutyCycle = 200; // Torsi awal (Starting Torque)
    digitalWrite(ledPin, HIGH);
    Serial.println("BLYNK: Motor Dinyalakan");
  } else {
    isMotorRunning = false;
    digitalWrite(ledPin, LOW);
    Serial.println("BLYNK: Motor Dimatikan");
  }
  applyMotor();
}

// Menerima perintah dari Slider Kecepatan di HP (Datastream V2)
BLYNK_WRITE(V2) {
  dutyCycle = param.asInt(); // Baca nilai slider (0-255)
  Serial.print("BLYNK: PWM Diubah menjadi ");
  Serial.println(dutyCycle);

  // Jika motor sedang ON, terapkan perubahan kecepatan ke alat
  if (isMotorRunning) {
    applyMotor();
  }
}

// --- SETUP & LOOP UTAMA ---
void setup() {
  Serial.begin(115200);

  // Setup Pin & PWM
  pinMode(ledPin, OUTPUT);
```

```
  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  ledcAttach(enable1Pin, freq, resolution);

  // Pastikan motor mati di awal
  applyMotor();

  Serial.println("Menghubungkan ke Wi-Fi dan Blynk Server...");
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  Serial.println("Berhasil Terhubung!");
}

void loop() {
  Blynk.run(); // Menjaga koneksi ke server tetap hidup
}
```

Penjelasan program

Program ini menandai transisi signifikan dari arsitektur eksekusi prosedural menuju arsitektur *event-driven* yang difasilitasi oleh peladen (*server*) Blynk Cloud. Fungsi iteratif loop() kini direduksi secara esensial menjadi instruksi tunggal, yakni Blynk.run(). Perintah ini bertugas mengeksekusi proses latar belakang (*background process*) guna menjaga stabilitas koneksi protokol TCP/IP (*keep-alive connection*) antara perangkat keras dan peladen tanpa adanya penundaan. Melalui pendekatan ini, mikrokontroler tidak lagi melakukan penguraian teks (*string parsing*) secara manual, melainkan secara pasif menanti interupsi instruksi dari pengguna melalui makro pendengar (*listener*) BLYNK_WRITE().

Komunikasi antara antarmuka pengguna di *smartphone* dan mikrokontroler direalisasikan secara terstruktur melalui *Datastream* atau jalur data maya. Blok BLYNK_WRITE(V3) bertindak sebagai pendengar untuk sakelar utama motor (ON/OFF). Ketika sakelar diaktifkan, program akan menyuntikkan torsi mula secara otomatis (dutyCycle = 150) jika kecepatan sebelumnya berada di angka nol, guna memecah gaya gesek statis (*static friction*) motor. Sementara itu, blok BLYNK_WRITE(V2) bertugas menangkap pergeseran nilai pada *slider* secara waktu nyata (*real-time*) untuk memperbarui variabel kecepatan. Kedua interupsi ini kemudian bermuara pada pemanggilan fungsi sentral applyMotor() yang menerjemahkan variabel tersebut menjadi sinyal fisik *Pulse Width Modulation* (PWM) beresolusi 8-bit menuju *driver* motor (Pin 12).

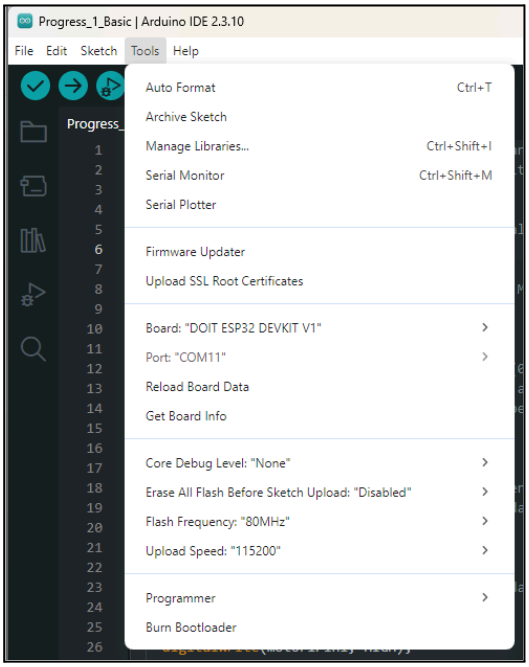
Sistem ini juga dilengkapi dengan kemampuan umpan balik visual (*visual feedback loop*) terpadu untuk mengirimkan status operasional secara dua arah (*bidirectional*) kembali ke layar pengguna. Di dalam blok applyMotor(), instruksi Blynk.virtualWrite() akan memancarkan nilai PWM yang sedang dieksekusi menuju indikator *gauge* pada *Virtual Pin* V1. Untuk keperluan representasi kecepatan putaran

mesin pada *Virtual Pin* V0, sistem memanggil fungsi matematis bawaan C++ yakni map() guna mengonversi rentang resolusi PWM (0–255) secara dinamis menjadi estimasi simulasi berskala 0 hingga 13.000 RPM. Apabila motor dimatikan secara instan melalui instruksi V3, sistem langsung mengimplementasikan rutinitas pembersihan data dengan memaksa nilai komponen indikator V0 dan V1 kembali ke titik nol, sehingga antarmuka aplikasi selalu sinkron dan selaras dengan status aktual perangkat keras.

2.6 Hasil Pengujian & Analisis

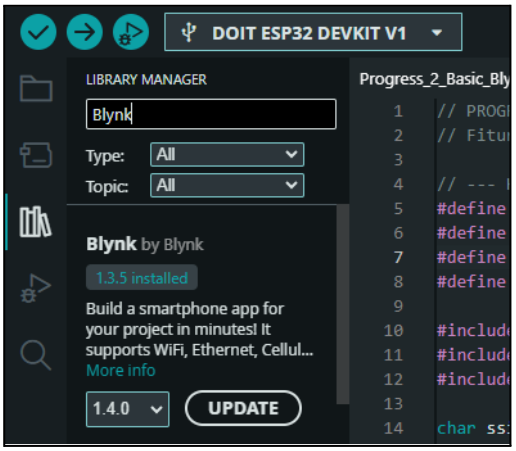
a. Proses Kompilasi dan Upload Program

Sebelum melakukan kompilasi dan *upload* program, pastikan terlebih dahulu bahwa *port* komunikasi sudah terhubung dengan benar ke kit ESP32. Pastikan jenis *board* yang dipilih adalah DOIT ESP32 DEVKIT V1 dengan kecepatan unggah (*Upload Speed*) sebesar 115200.



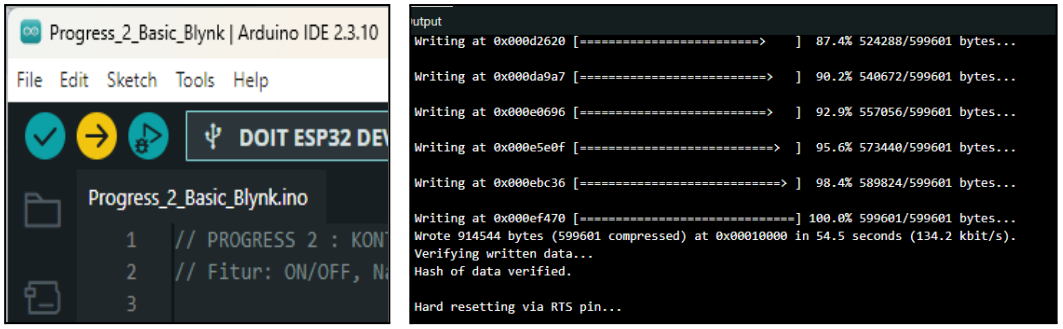
Gambar 19. Menu Tools Pemilihan Board dan Port pada Arduino IDE

Selain itu, pastikan kembali bahwa *library* "Blynk by Blynk" telah terinstal pada *Library Manager*, sesuai dengan tahap persiapan (*setup*).



Gambar 20. Pengecekan Library Blynk pada Arduino IDE

Hal krusial lainnya yang wajib dipastikan adalah jaringan Wi-Fi (atau *hotspot* seluler) yang digunakan untuk mengoneksikan ESP32 beroperasi pada pita frekuensi 2,4 GHz. Modul nirkabel pada ESP32 tidak mendukung konektivitas pada frekuensi 5 GHz. Jika pengaturan jaringan sudah dipastikan benar, langkah selanjutnya adalah menekan tombol *Upload* untuk menanamkan kode (*firmware*) ke dalam mikrokontroler.

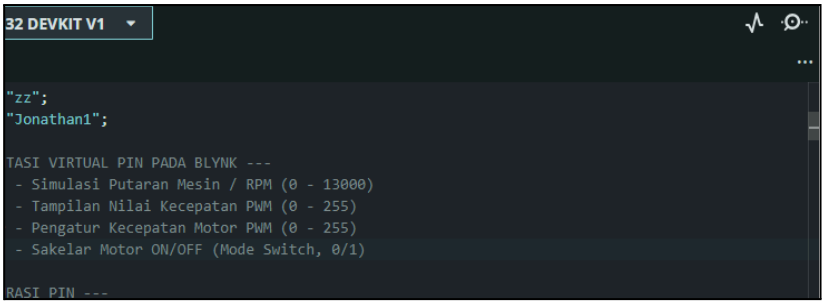


Gambar 21 dan 22. Proses Kompilasi
dan Upload Program

Tunggu hingga proses *upload* selesai. Jika kode berhasil diunggah tanpa galat (*error*), maka pada panel *Output* akan muncul log keterangan bahwa memori *flash* telah berhasil ditulis, ditandai dengan informasi "Wrote ... bytes" dan eksekusi *Hard resetting via RTS pin*.

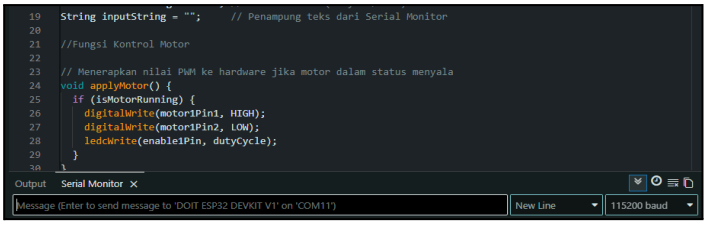
b. Pengujian Perintah via Serial Monitor & Blynk

Setelah program berhasil ditanamkan, pengujian fungsionalitas dan pemantauan (*monitoring*) dilakukan melalui antarmuka Serial Monitor. Langkah pertama adalah mengklik ikon berbentuk kaca pembesar di sudut kanan atas jendela Arduino IDE. Fungsi utama dari Serial Monitor ini adalah untuk memantau status operasional jaringan dan mengevaluasi komunikasi perangkat dengan *server* Blynk secara *real-time*.



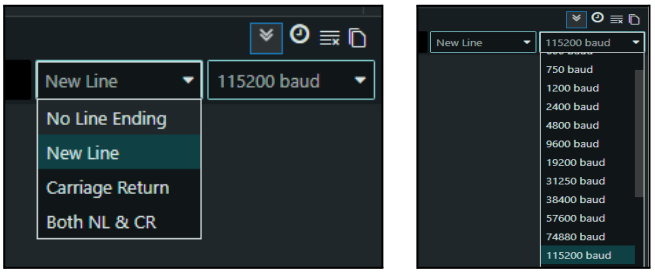
Gambar 23. Ikon Serial Monitor
pada Arduino IDE

Selanjutnya, jendela Serial Monitor akan terbuka pada bagian bawah layar (panel *Output*).



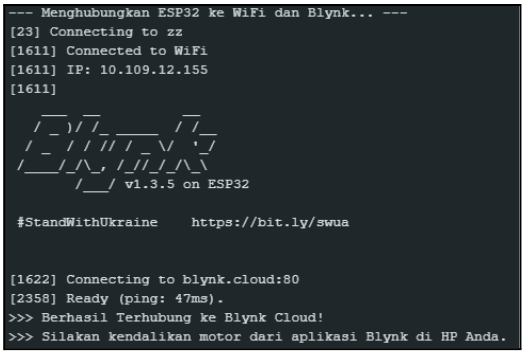
Gambar 24. Tampilan Jendela
Antarmuka Serial Monitor

Sebelum melakukan uji coba sinkronisasi antara perangkat keras dengan aplikasi Blynk, terdapat dua parameter komunikasi yang harus disesuaikan di pojok kanan jendela Serial Monitor. Pastikan pengaturan format baris (*Line Ending*) berada pada opsi New Line dan kecepatan komunikasi (*Baudrate*) berada pada angka 115200 baud, selaras dengan deklarasi Serial.begin(115200) pada fungsi *setup*.



Gambar 25 dan 26. Pengaturan Opsi Line
Ending dan Baudrate

Setelah konfigurasi *baudrate* sesuai, uji coba dapat langsung dilakukan menggunakan *Device* yang telah dikonfigurasi pada aplikasi Blynk. Sebelum memberikan instruksi dari ponsel, disarankan untuk menekan tombol EN (Enable/Reset) pada modul ESP32 secara fisik. Hal ini dilakukan untuk memicu proses *booting* ulang dan memantau log sinkronisasi jaringan pada Serial Monitor guna memastikan ESP32 berhasil terhubung.



Gambar 27. Log Hasil Pemantauan
Koneksi ke Server Blynk

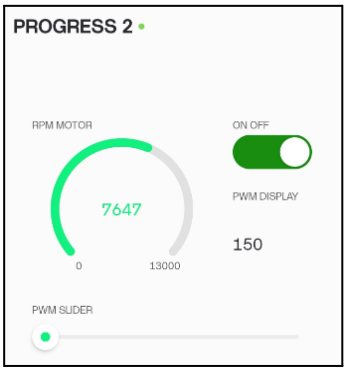
Buka aplikasi Blynk pada *smartphone*. Indikator visual keberhasilan koneksi *device* ditandai dengan perubahan ikon status perangkat di sudut kanan atas dari warna abu-abu (offline/offline) menjadi warna hijau.

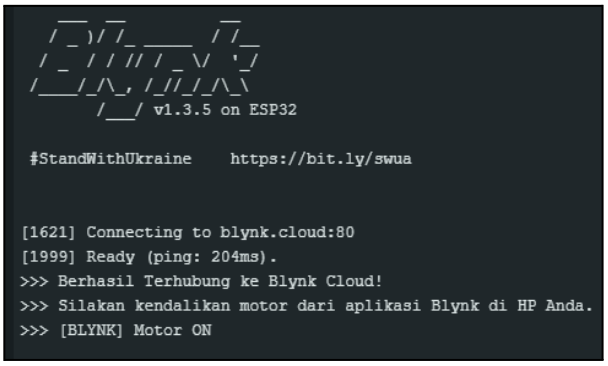




Gambar 28 dan 29. Perubahan Indikator
Status Perangkat Menjadi Online

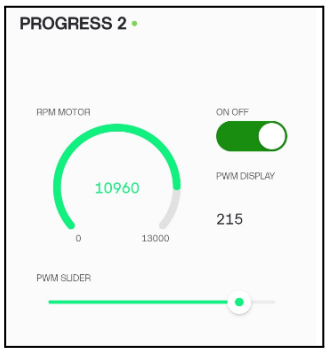
Pengujian operasional dimulai dengan menyalakan sakelar mesin pada aplikasi Blynk. Seketika tombol diaktifkan, antarmuka Serial Monitor mencetak notifikasi bahwa motor dihidupkan (ON), dan indikator *Gauge* pada Blynk secara otomatis melonjak ke angka PWM 150 (mekanisme pengungkit torsi awal / *starting torque*).





Gambar 30 dan 31. Umpan Balik Torsi
Awal Saat Motor Dihidupkan

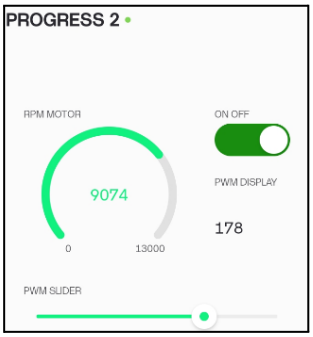
Pengujian dilanjutkan dengan menggeser *widget Slider* kecepatan (PWM SLIDER). Indikator kecepatan (PWM DISPLAY dan RPM MOTOR) secara responsif menampilkan perubahan angka. Perubahan nilai PWM ini juga langsung terekam pada log Serial Monitor Arduino IDE.

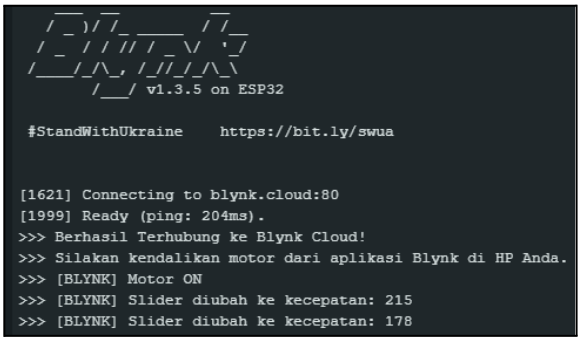




Gambar 32 dan 33. Sinkronisasi
Modifikasi Kecepatan (Slider)

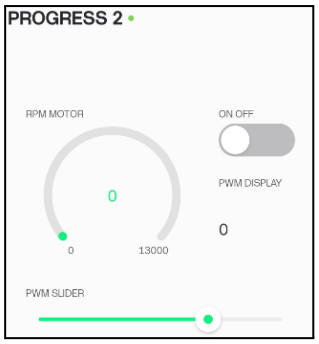
Saat nilai parameter kecepatan diturunkan melalui *Slider*, komponen *Gauge* memperbarui nilai variabel tersebut seketika secara waktu nyata (*real-time*). Log pada Serial Monitor mengonfirmasi bahwa instruksi penurunan nilai PWM telah berhasil ditangkap dan dieksekusi oleh mikrokontroler.





Gambar 34 dan 35. Responsivitas Penurunan
Variabel Kecepatan

Selanjutnya pengujian ditutup dengan mematikan motor melalui tombol *Switch* pada Blynk. Panel indikator di aplikasi seketika kembali ke nilai nol, dan Serial Monitor mencetak notifikasi penyelesaian proses (Motor OFF).





Gambar 36 dan 37. Eksekusi Penghentian
Rotasi Motor

c. Analisis Keseluruhan

Berdasarkan serangkaian pengujian lapangan yang telah dilakukan, sistem kendali motor nirkabel berbasis *platform* Blynk terbukti beroperasi dengan tingkat presisi dan reliabilitas yang sangat tinggi. Modul mikrokontroler ESP32 menunjukkan kapabilitas yang konsisten dalam melakukan negosiasi jaringan pada pita 2,4 GHz dan melakukan otentikasi protokol TCP/IP menuju *server cloud*.

Kecepatan transmisi instruksi dari antarmuka *smartphone* menuju aktuator pergerakan fisik (driver L298N) terjadi secara hampir instan (*real-time*) tanpa adanya *delay* yang mengganggu. Paradigma komunikasi *event-driven* yang memanifestasikan fungsi *Virtual Pin* (Datastream) terbukti mampu memberikan sinkronisasi data sirkuler dua arah (*bidirectional feedback loop*) yang sangat stabil; hal ini direpresentasikan oleh keselarasan angka *Gauge* PWM (V1) dan estimasi RPM (V0) di aplikasi genggam dengan status tegangan riil pada motor keras. Jika dikomparasikan dengan skema komunikasi Serial konvensional (berbasis kabel) pada Progress 1, sistem ekosistem IoT pada Progress 2 ini berhasil memecahkan kendala limitasi ruang, memungkinkan perangkat keras untuk ditempatkan dan dikendalikan dari lokasi geografis mana pun selama infrastruktur jaringan internet tersedia.